QD-Evolve

pypi v0.1.12 中文版

Multi-agent AI framework with A2A protocol support, group chat, persistent memory, and extensible tool system.

- DESIGN.md — design philosophy, invariants, architecture, implementation

Installation

Prerequisites

- **Python 3.13+** — download
- **Mosquitto v5 broker** (MQTT/GChat mode only) — download

Verify Python is ready:

Windows

```
python --version
# Python 3.13.x  ← must be 3.13 or newer
```

If the command is not found: 1. Re-run the Python installer 2. Check **"Add python.exe to PATH"** at the bottom of the first screen 3. Or search "Manage app execution aliases" in Windows Settings and turn off the Python alias that opens the Store

macOS / Linux

```
python3 --version
# Python 3.13.x  ← must be 3.13 or newer
```

Step 1 — Create a project folder

```
mkdir my-agent
cd my-agent
```

Step 2 — Create and activate a virtual environment

Windows

```
python -m venv .venv
.venv\Scripts\activate
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

Step 3 — Install qd-evolve

```
pip install qd-evolve --extra-index-url https://abetlen.github.io/llama-cp

# Optional extras
pip install qd-evolve[memory] --extra-index-url https://abetlen.github.io/
pip install qd-evolve[boat] --extra-index-url https://abetlen.github.io/ll
```

> If you use uv instead of pip, the extra index is configured automatically — just run `uv add qd-evolve`.

Step 4 — Initialize your project

```
qd-evolve init
```

This copies default tools, skills, and config templates into your project folder:

```
my-agent/
├── .venv/                  # Virtual environment
├── tools/                  # User tools - add/delete freely
│   ├── bridge/             #   Bridge connectors (OAT, MCP)
│   ├── cli/                #   CLI tool wrappers
│   ├── func/               #   Python function tools
│   └── mcp/                #   MCP server configs
├── skills/                 # Skills - add/delete freely
│   ├── baidu-search/
│   ├── register-cli/
│   ├── search-tools/
│   └── ...
```

```
├── config.minimal.json    # Minimal config — copy to config.json
└── config.json.example    # Full config reference
```

Running `init` again is safe: existing files are never overwritten. New default files from package updates are added.

Step 5 — Configure

```
copy config.minimal.json config.json    # Windows
# cp config.minimal.json config.json     # macOS / Linux
```

Edit `config.json` and set your API key. Minimal setup for single-agent chat:

```
{
  "env_vars": {
    "PYTHONIOENCODING": "utf-8",
    "PYTHONUTF8": "1",
    "SERPER_API_KEY": "YOUR_SERPER_API_KEY"
  },
  "default_provider": "deepseek",
  "default_model": "deepseek-v4-pro",
  "providers": [
    {
      "name": "deepseek",
      "api_key": "YOUR_DEEPSEEK_API_KEY",
      "base_url": "https://api.deepseek.com",
      "api": "openai-completions",
      "models": [
        { "name": "deepseek-v4-pro", "reasoning": true, "context_window":
      ]
    }
  ],
  "agents_config": {
    "agents": [
      {
        "name": "default",
        "toolbox": {
          "tools": {
            "load_func": "preload",
            "load_skill": "preload",
            "load_cli": "preload"
          }
        }
      }
    ]
  }
}
```

> **API keys you'll need:** - **Provider key** (DeepSeek, OpenAI, Anthropic, etc.) — required for chat - **Serper key** — optional, for web search. Free tier at serper.dev. Without it the agent may open browser windows to search.
>
> See Configuration below for full multi-agent, MQTT, and toolbox setup.

Verify

```
qd-evolve chat --agent default
```

Troubleshooting

`pip install` fails with CMake / nmake errors

This means pip is trying to build `llama-cpp-python` from source. Make sure you included `--extra-index-url`:

```
pip install qd-evolve --extra-index-url https://abetlen.github.io/llama-cp
```

Browser windows open when the agent searches the web

The agent has a search tool that needs a Serper API key (`SERPER_API_KEY` in `env_vars`). Without it, the agent may fall back to the browser MCP tool which opens visible browser windows. Get a free key at serper.dev and add it to `config.json`.

`'python' is not recognized` or `python: command not found`

Python is not installed or not on your PATH. See the verification steps in Prerequisites above.

Install from source

```
git clone https://github.com/juzcn/qd-evolve
cd qd-evolve

# Install dependencies
uv sync

# Optional: BOAT bridge extras
uv sync --extra boat
```

Source installs have `tools/` and `skills/` already in the project root — no `init` needed.

Quick Start

```
# Single-agent chat
qd-evolve chat --agent default

# Multi-agent A2A chat over HTTP
qd-evolve a2a-http

# Run an agent as standalone A2A HTTP server
qd-evolve a2a-http serve --agent <name>

# Multi-agent in-process chat (all agents loaded locally, no network)
qd-evolve a2a-inproc

# Multi-agent MQTT chat (requires Mosquitto v5 broker)
qd-evolve a2a-mqtt

# Run an agent as MQTT-accessible server
qd-evolve a2a-mqtt serve --agent <name>

# Group chat — WeChat-style multi-agent group (requires Mosquitto v5 broke
# Supports: AI agents, terminal human agents, WeChat human agents
qd-evolve gchat --agent <name>

# Manage tool enable/disable/preload
qd-evolve toolbox --agent <name>

# Browse and search conversation memories
qd-evolve memory --agent <name>
```

Four Systems

System	Entry	Transport	Use Case
Chat	`qd-evolve chat --agent <name>`	In-process only	Single-agent, no network
A2A Inproc	`qd-evolve a2a-inproc`	In-process only	Multi-agent in-process

System	Entry	Transport	Use Case
A2A	`qd-evolve a2a-http`	HTTP + in-proc	Multi-agent over HTTP/SSE
MQTT	`qd-evolve a2a-mqtt`	MQTT v5 + in-proc	Multi-agent over MQTT
GChat	`qd-evolve gchat`	MQTT v5 (group topics)	WeChat-style group chat

Each system is fully independent — no protocol fallback between them. See DESIGN.md for the full architecture.

Configuration

All configuration via `config.json`. No CLI config commands, no `.env` files.

Provider & Model

```
{
  "default_provider": "openai",
  "default_model": "gpt-4o",
  "providers": [
    {
      "name": "openai",
      "api_key": "sk-...",
      "base_url": "https://api.openai.com/v1",
      "api": "openai-completions",
      "models": [
        { "name": "gpt-4o", "context_window": 128000, "max_tokens": 4096 }
      ]
    },
    {
      "name": "anthropic",
      "api_key": "sk-ant-...",
      "api": "anthropic",
      "models": [
        { "name": "claude-sonnet-4-6", "context_window": 200000, "max_toke
      ]
    },
    {
      "name": "deepseek",
      "api_key": "...",
      "base_url": "https://api.deepseek.com",
```

```
      "api": "openai-completions",
      "models": [
        { "name": "deepseek-v4-pro", "reasoning": true, "context_window":
      ]
    }
  ]
}
```

Three API types: `openai-completions`, `openai-response`, `anthropic`. Set at provider level via `api` field. Streaming is global (`stream` field). Reasoning/thinking is per-model (`reasoning: true`).

Multi-Agent

```
{
  "agents_config": {
    "chat_agent": "planner",
    "agents": [
      {
        "name": "planner",
        "description": "Plans and delegates tasks",
        "provider": "openai",
        "model": "gpt-4o",
        "memory_db": "planner.db",
        "server": { "host": "127.0.0.1", "port": 8001 },
        "toolbox": { "tools": {} }
      },
      {
        "name": "human",
        "description": "Human for approvals",
        "provider": "human",
        "server": { "host": "127.0.0.1", "port": 8002 }
      },
      {
        "name": "wechat_user",
        "description": "Human via WeChat iLink",
        "provider": "wechat-human",
        "server": { "host": "127.0.0.1", "port": 8003 }
      }
    ]
  }
}
```

Per-agent provider/model with global fallback. `provider: "human"` for terminal human agents, `"wechat-human"` for WeChat iLink bridge. Each agent has its own memory DB, server config, and toolbox state. WeChat human agents persist their session token via the `wechat_session` field.

MQTT Broker

```
{
  "agents_config": {
    "mqtt_broker": {
      "host": "127.0.0.1",
      "port": 1883
    }
  }
}
```

Requires external Mosquitto v5 broker.

Toolbox

Tool enable/disable/preload per agent, managed via `qd-evolve toolbox --agent <name>` (Textual TUI) or by editing `config.json` directly.

```
{
  "agents_config": {
    "agents": [{
      "name": "planner",
      "toolbox": {
        "tools": { "run_shell": "preload", "web_search": "enabled" },
        "mcp_servers": { "filesystem": "disabled" },
        "bridge": { "oat:boat": "enabled", "oat:coat": "disabled" },
        "cli": { "git": "preload" },
        "skills": { "code-review": "preload" }
      }
    }]
  }
}
```

Three states: `enabled` (on-demand schema), `preload` (schema at startup), `disabled` (invisible to agent).

Runtime Features

Slash Commands

Command	Description
`/models`	Switch provider/model

Command	Description
`/agents`	Switch agent
`/tools`	List tools
`/load <tool>`	Load tool schema
`/memory`	List saved memories with full content
`/compress`	Force context compression
`/clear`	Clear conversation
`/help`	Show help
`/quit`	Exit

Heartbeat

Agent-managed idle detection. When no activity for `heartbeat_idle_seconds`, sends a heartbeat prompt to the LLM. If LLM responds with `"."`, stays silent. Set `0` to disable. Mode-specific templates selected automatically.

Replay Mode

`--replay <file>` feeds pre-recorded inputs for automated testing. `--output <file>` captures output.

Token Stats

Per-turn and cumulative input/output tokens with context window usage percentage.

A2A Protocol

Full A2A v1.0 implementation:

- **Agent discovery**: `/.well-known/agent.json`
- **Methods**: `message/send`, `message/stream`, `tasks/get`, `tasks/cancel`, `tasks/resubscribe`, `tasks/pushNotification`, `agent/getExtendedAgentCard`
- **Task lifecycle**: submitted → working → completed / failed / canceled / input_required

- **SSE streaming**: `message/stream` returns `StreamResponse` events
- **Push notifications**: webhook callbacks on task completion

Group Chat

WeChat-style multi-agent group via MQTT. All configured agents form a single group.

- **AI agents**: Background loop processes `@mentions`, runs agent in parallel, publishes responses
- **Terminal human agents** (`provider: "human"`): Interactive prompt — type messages, see group activity
- **WeChat human agents** (`provider: "wechat-human"`): Bidirectional WeChat iLink bridge — long-poll for incoming WeChat messages, forward group responses back to WeChat. QR login on startup, session persisted to `config.json`
- `@all` mentions everyone; specific `@agent_name` directs to one agent

Project Layout

```
qd-evolve/
├── qd_evolve/       # Main package (agent, core, tools, utils, _templates
├── tools/           # User tools (func, cli, mcp, bridge)
├── skills/          # Skills (SKILL.md files)
├── templates/       # User Jinja2 template overrides
├── tests/           # pytest suite
├── config.json      # All configuration
├── memory.db        # Conversation memory (SQLite + sqlite-vec)
└── pyproject.toml   # Dependencies and build config
```

See DESIGN.md for architecture and full module map.

Requirements

- Python 3.13+
- External Mosquitto v5 broker (MQTT/GChat mode only)
- API keys for configured providers (DeepSeek, OpenAI, Anthropic, etc.)

License

MIT